

Instituto Tecnológico de Costa Rica

Algoritmos y estructuras de datos 1

Proyecto 1; Monopoly

Cristopher Viquez Quirós 2026014559

Jimena Méndez Campos 2025125035

Felipe Esteban Sanabria Pacheci 2026089891

02 de octubre de 2026

- **Cliente**

Descripción General

La capa cliente del proyecto Monopoly TEC es la encargada de gestionar la interacción directa con el usuario, procesar eventos de la interfaz gráfica y por consola, controlar los componentes de hardware (sensor RFID y lector de datos) y mantener la comunicación en tiempo real con el servidor mediante un socket TCP.

El módulo cliente se divide internamente en dos componentes principales:

1. **Cliente Lógico (Client):** Administra el estado local del jugador, las llamadas al módulo de red TCP y la comunicación con el hardware.
2. **Interfaz Gráfica / Consola (ClientGUI y Program):** Presenta de forma visual el tablero de juego (cuadrícula 11x11), las posiciones de los jugadores, los mensajes de evento y los controles para la toma de decisiones durante el turno.

Arquitectura de Clases y Responsabilidades Client.Cliente

Es la clase central de la lógica del cliente. Actúa como intermediario entre la capa de red (ClienteTcp), el hardware y la interfaz de usuario.

- **Mantenimiento del Estado Local:** Mantiene una copia actualizada de la lista de jugadores (ListaSimple<JugadorEstado>) y del historial de transacciones (ListaSimple<TransaccionInfo>).
- **Sincronización:** Recibe los mensajes entrantes desde el servidor TCP, deserializa los datos según el TipoMensaje y dispara eventos de C# (OnEvento, OnEstadoActualizado, OnError, OnOfertaCompra, OnResultadoDado, etc.) para notificar a la interfaz gráfica.
- **Envío de Acciones:** Empaqueta y envía peticiones al servidor para acciones del juego como tirar dados, comprar o rechazar propiedades, pagar deudas, registrar/validar tarjetas RFID y finalizar el turno.

Client.DispositivoHardware

Se encarga de la integración y lectura de los dispositivos de hardware conectados al equipo mediante puerto serial (UART / USB COM).

- **Lectura Asíncrona:** Ejecuta un hilo secundario (Thread) dedicado a escuchar continuamente las líneas de texto enviadas por el Arduino a una velocidad de 9600 baudios.
- **Procesamiento de Comandos:**
 - **Dados:** Decodifica tramas con el formato DADOS:x,y (donde x e y representan los valores de los dados) y dispara el evento DatosLanzados.
 - **RFID:** Decodifica tramas con el formato RFID:id (identificador único de la tarjeta o llavero leído) y dispara el evento TarjetaLeida.

ClientGUI.TableroControl

Es un control personalizado derivado de System.Windows.Forms.Panel encargado del renderizado visual interactivo del tablero de juego.

- **Distribución Gráfica:** Mapea linealmente las 40 casillas del juego a lo largo del perímetro de una cuadrícula de 11×11 celdas. La casilla de "Salida" (posición 0) se ubica en la esquina inferior derecha.
- **Renderizado:** Dibuja dinámicamente las imágenes o nombres de las casillas, franjas de color correspondientes a los grupos de propiedades, indicadores del dueño de cada casilla y las fichas de los jugadores mediante un ordenamiento visual dentro de cada casilla.
- **Representación de Dados:** Renderiza visualmente en la zona central del tablero los dados lanzados.

ClientGUI.DatosTablero

Clase estática auxiliar de configuración que centraliza el modelo de datos visual del tablero.

- Almacena los nombres de las 40 casillas, la categorización por grupos de color (Marrón, Celeste, Rosa, Naranja, Rojo, Amarillo, Verde, Azul, Asociaciones y Especiales) y los colores asignados a las fichas de los jugadores (Rojo, Azul, Verde y Morado).

ClientGUI.VentanaPrincipal y Client.Program

Representan los puntos de entrada del cliente. VentanaPrincipal proporciona la experiencia GUI mediante Windows Forms, mientras que Program ofrece una alternativa por consola.

- **Administración de Pantallas:** Controla la transición entre la pantalla de conexión (ingreso de IP, nombre y puerto serial COM) y la pantalla de la partida.
- **Manejo de Hilos (Delegación GUI):** Debido a que la lectura de red y de hardware ocurre en hilos independientes, la interfaz gráfica utiliza Invoke / BeginInvoke para actualizar los elementos de la interfaz de manera segura sin bloquear el hilo principal.
- **Simulación:** Permite continuar con las pruebas del sistema activando controles simulados de dados y tarjetas RFID cuando no se cuenta con el hardware físico conectado al puerto serial.

Flujo de Comunicación y Funcionamiento

1. **Conexión e Identificación:** El cliente ingresa la IP del servidor y su nombre. Envía un mensaje CONECTAR. Al ser aceptado, el servidor le asigna un ID de jugador y la cantidad de casillas del tablero.
2. **Registro de Hardware / RFID:** En la sala de espera predeterminada, los jugadores deben registrar su tarjeta RFID física o simulada mediante la lectura por puerto serial.
3. **Desarrollo del Turno:**

o Al recibir la notificación TU_TURN0, la interfaz habilita las opciones de interacción. o El jugador presiona el botón físico en el módulo para lanzar los dados. DispositivoHardware lee la trama del puerto serial y Cliente transmite los valores al servidor. o Ante eventos como caer en una propiedad vacía, el servidor responde con una oferta (OFERTA_COMPRA), solicitando la confirmación de compra o rechazo mediante la tarjeta RFID.

4. **Cierre e Historial:** El cliente puede solicitar en cualquier momento la consulta del historial de transacciones registradas o desconectarse limpiamente del servidor.

- **CienteTCP**

Descripción General

La capa **Network** define el protocolo de comunicación y la infraestructura de socket TCP necesaria para permitir la interacción bidireccional y asíncrona en tiempo real entre el servidor central y los clientes del juego **Monopoly TEC**.

Esta capa abstrae la transmisión de datos serializando los mensajes en formato **JSON** con delimitador de salto de línea (\n), lo que garantiza la entrega ordenada y libre de fragmentación de cada paquete de red.

Componentes y Arquitectura de Clases Network.CienteTcp

Clase encargada de administrar el socket TcpClient y el flujo de datos (NetworkStream) en el lado del cliente.

- **Establecimiento de Conexión (Conectar):** Inicia la conexión síncrona hacia la dirección IP y puerto del servidor. Al conectarse exitosamente, arranca un hilo secundario (Background Thread) para escuchar el canal de entrada de forma continua sin bloquear el hilo de la interfaz gráfica.
- **Procesamiento y Acumulación de Tramas (EscucharServidor):**
 - Lee el flujo de bytes en un buffer de 4 KB y utiliza un StringBuilder para acumular los fragmentos de texto entrantes.
 - Resuelve el problema de fragmentación e hiper-empaquetamiento de TCP buscando activamente el carácter salto de línea (\n). o Extrae cada línea completa, la deserializa mediante Mensaje.Deserializar y dispara el evento MensajeRecibido.
- **Envío de Mensajes (Enviar):** Serializa la instancia de Mensaje, concatena el delimitador \n, la convierte a codificación **UTF-8** y la transmite por el NetworkStream.
- **Manejo de Desconexión (Cerrar):** Se encarga de cerrar de manera limpia el socket y el stream de datos, disparando el evento Desconectado.

Protocolo de Comunicación (Protocolo.cs)

El protocolo de red define la estructura estandarizada de los mensajes e intercambios de datos entre nodos.

Clase Mensaje

Representa el envoltorio genérico (wrapper) para todas las comunicaciones del sistema.

- **Tipo (TipoMensaje):** Enumeración que especifica la acción u orden del mensaje.
- **Datos (object?):** Contenedor genérico para la carga útil (*payload*).
- **LeerDatos<T>():** Método genérico que permite deserializar dinámicamente la propiedad Datos al tipo de *payload* específico que corresponde al TipoMensaje.

Enumeración TipoMensaje

Divide los tipos de comandos según la dirección de la transmisión:

Dirección	TipoMensaje	Descripción / Propósito
Cliente → Servidor	CONECTAR	Solicita el ingreso a la partida con un nombre de jugador (DatosConectar).
	TIRAR_DADO	Envía el resultado físico de los dados leídos por el hardware (DatosTirarDado).
	COMPRAR_PROPIEDAD	Confirma la compra de la propiedad donde cayó el jugador.
	NO_COMPRAR	Rechaza la oferta de compra de la propiedad actual.
	PAGAR_DEUDA	Solicita el pago de deudas pendientes (rentas o impuestos).
	TERMINAR_TURNO	Finaliza formalmente el turno del jugador actual.
	CONSULTAR_TRANSACCIONES	Solicita la lista histórica de movimientos financieros.

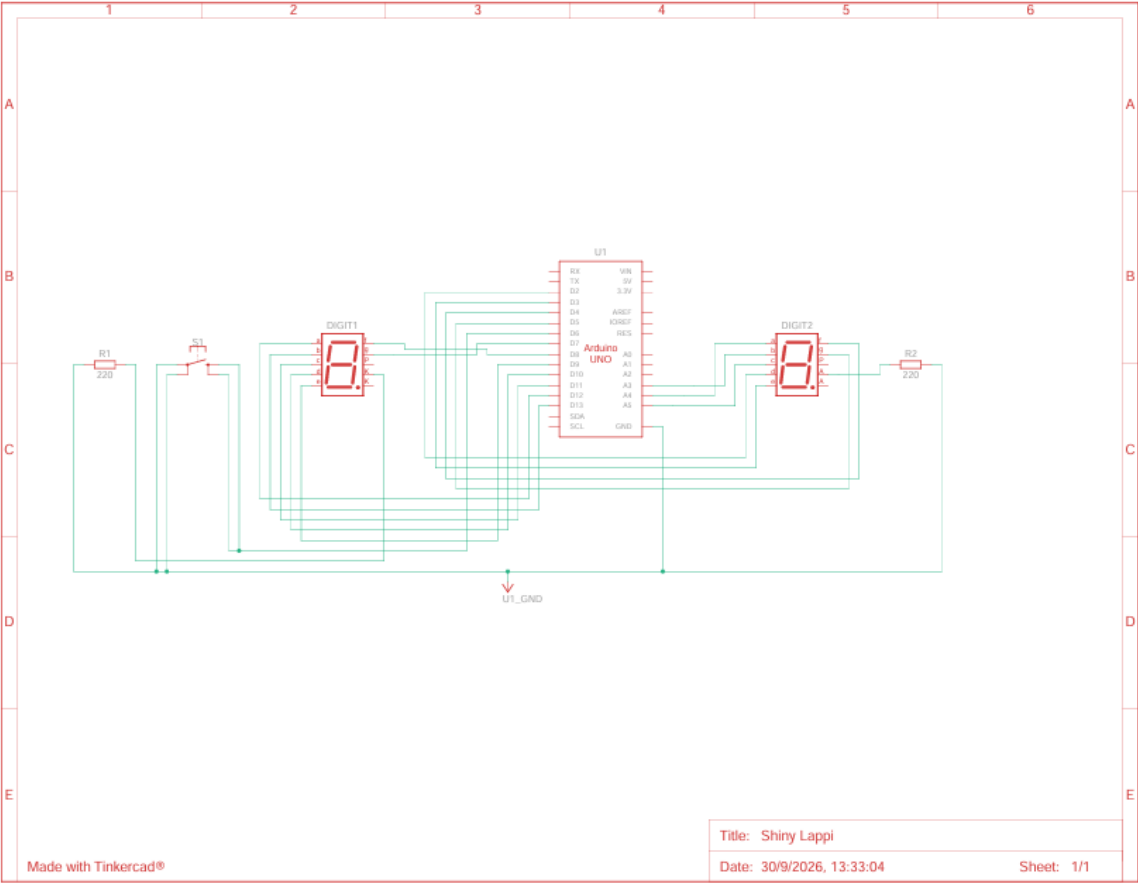
Dirección	TipoMensaje	Descripción / Propósito
	REGISTRAR_TARJETA	Vincula una tarjeta RFID a un jugador en la fase de registro (DatosTarjeta).
	TARJETA_RFID	Notifica la lectura de la tarjeta RFID durante el juego (DatosTarjeta).
	DESCONECTAR	Notifica la desconexión voluntaria del jugador.
Servidor → Cliente	CONEXION_ACEPTADA	Confirma el ingreso y envía el estado inicial (DatosConexionAceptada).
	CONEXION_RECHAZADA	Rechaza el ingreso explicando el motivo (DatosConexionRechazada).
	ACTUALIZAR_ESTADO	Transmite el estado global actualizado de todos los jugadores (DatosActualizarEstado).
	TU_TURN0	Notifica al cliente especificado que es su turno de jugar (DatosTuTurno).

Dirección	TipoMensaje	Descripción / Propósito
	RESULTADO_DADO	Notifica a todos los clientes el valor obtenido en los dados (DatosResultadoDado).
	OFERTA_COMPRA	Notifica que una casilla está disponible para compra (DatosOfertaCompra).
	HISTORIAL_TRANSACCIONES	Devuelve el listado de transacciones solicitadas (DatosHistorialTransacciones).
	EVENTO	Difunde descripciones de sucesos del juego para la bitácora (DatosEvento).
	ERROR	Informa un error de validación o regla de juego (DatosError).
	FIN_JUEGO	Declara el fin de la partida y anuncia al ganador (DatosFinJuego).

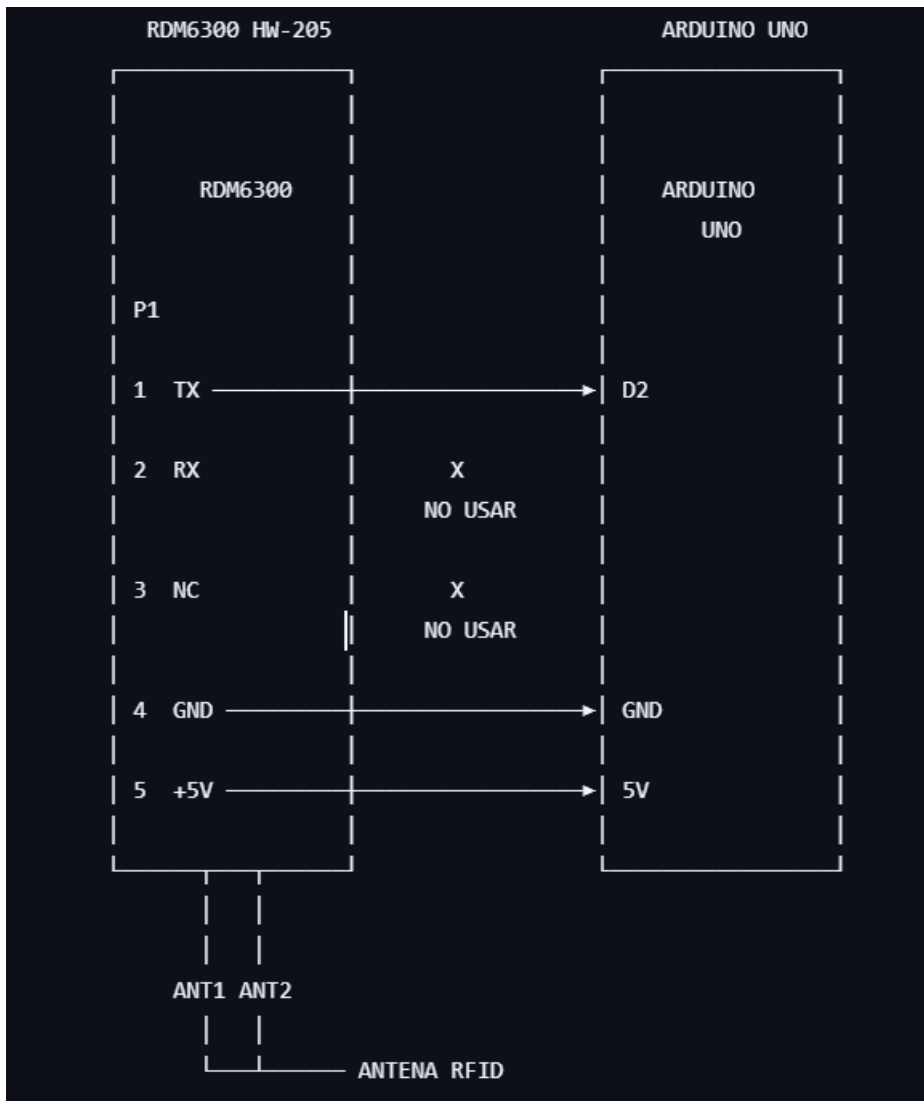
Estructuras de Datos / *Payloads* Destacados

- **JugadorEstado:** Objeto DTO que transporta la información de cada jugador por la red (ID, Nombre, Posición, Saldo, lista de IDs de propiedades, estado de bancarrota y registro de tarjeta RFID).
- **TransaccionInfo:** Estructura de transferencia de datos para la bitácora financiera que viaja entre el servidor y el cliente (ID, FechaHora, Turno, Tipo, Jugadores origen/destino, Monto y Descripción).

- Diagrama display



- Diagrama lector



- **Estructuras**

Introducción y Justificación General

Para el desarrollo del sistema de Monopoly Distribuido, se implementaron estructuras de datos lineales propias desde cero en el lenguaje C#, respetando la restricción de no utilizar las colecciones genéricas del sistema (como List<T>, LinkedList<T>, Queue<T>, etc.).

La arquitectura interna se basa en el concepto de Nodos Genéricos (NodoSimple<T> y NodoDoble<T>), lo cual garantiza una alta reutilización de código, desacoplamiento de la lógica del juego y tipado fuerte (type safety).

Descripción Detallada de las Estructuras

1. Tablero de Juego: Lista Circular Doblemente Enlazada (ListaCircularDoble<T>)

- **Definición y Estructura:** Es una estructura de datos lineal compuesta por nodos doblemente enlazados (NodoDoble<T>), donde cada nodo posee un puntero hacia el nodo Siguiente y otro hacia el Anterior. Su característica distintiva es que el último nodo se enlaza directamente con el primer nodo (Cabeza), formando una cadena cerrada o bucle.
- **Nodo Asociado:** NodoDoble<Casilla> (contiene la casilla del tablero).
- **Justificación de Uso en el Proyecto:** El tablero de Monopoly consta de 24 casillas. Una lista circular doble es ideal por tres razones principales:
 1. **Naturaleza Ciclica del Tablero:** Permite que los jugadores den múltiples vueltas continuas al tablero de forma fluida. Al pasar por la última casilla, el puntero Siguiente redirige de inmediato a la casilla de Inicio sin requerir reinicios manuales de índices.
 2. **Desplazamiento Bidireccional:** Permite que las cartas de eventos u otras dinámicas muevan a los jugadores tanto hacia adelante (usando Siguiente) como hacia atrás (usando Anterior).

- **Operaciones Principales e Implementación:**
 - o **Agregar(T dato):** Inserta una casilla al tablero manteniendo los enlaces circulares y bidireccionales en $O(1)$.
 - o **Mover(NodoDoble<T> nodoInicio, int pasos):** Recibe la posición actual del jugador y la suma de los dados. Realiza iteraciones hacia adelante ($pasos > 0$) o hacia atrás ($pasos < 0$) retornando el nodo destino exacto.

2. Administración de Turnos: Cola Circular (ColaCircular<T>)

- **Definición y Estructura:** Es una variante de la cola implementada internamente sobre nodos simples (NodoSimple<T>). Mantiene una referencia al puntero ultimo, cuyo puntero ultimo.Siguiente apunta siempre al frente de la cola (el primer jugador en la secuencia).
- **Nodo Asociado:** NodoSimple<Jugador>.
- **Justificación de Uso en el Proyecto:** El sistema de turnos exige una administración estrictamente secuencial y continua entre los 4 jugadores activos:
 1. **Garantía de Turno:** Impide que los jugadores actúen fuera de su turno asignado, respondiendo a las validaciones del servidor.
 2. **Rotación Eficiente:** Cuando un jugador finaliza su turno, la estructura no elimina al jugador, sino que simplemente desplaza el puntero del turno activo hacia el siguiente participante en $O(1)$.
- **Operaciones Principales e Implementación:**
 - o **Encolar(T dato):** Registra a los jugadores al iniciar la partida en $O(1)$.
 - o **AvanzarTurno():** Rota la referencia del puntero al siguiente nodo ($ultimo = ultimo.Siguiente$), permitiendo reiniciar el ciclo de turnos de manera limpia y sin reconstruir la estructura.
 - o **VerSiguiente():** Retorna el jugador que posee el turno actual sin modificar el estado de la cola.

3. Historial de Transacciones: Lista Doblemente Enlazada (ListaDoble<T>)

- **Definición y Estructura:** Es una lista lineal constituida por nodos doblemente enlazados (NodoDoble<T>) que mantiene dos referencias explícitas: el nodo inicial (Cabeza) y el nodo final (Cola). A diferencia de la lista circular, sus extremos no se conectan entre sí (los punteros inicial y final son null).
- **Nodo Asociado:** NodoDoble<Transaccion>.
- **Justificación de Uso en el Proyecto:** El pliego de requerimientos exige almacenar de forma cronológica todas las operaciones económicas (compras, alquileres, eventos, premios) y permitir lecturas completas en ambos sentidos:

Lectura desde la más antigua: Se realiza un recorrido natural desde Cabeza hasta Cola.

Lectura desde la más reciente: Gracias al método

ObtenerEnOrdenInverso() y los enlaces Anterior, se puede recorrer la lista comenzando en Cola en dirección a Cabeza, facilitando la consulta de los últimos movimientos realizados en la partida.

Exportación a Reportes: Permite iterar fácilmente toda la bitácora económica para generar el archivo de reporte .TXT obligatorio.

Operaciones Principales e Implementación:

- **Agregar(T dato):** Añade un nuevo registro de transacción al final del historial en $O(1)$.
- **ObtenerEnOrdenInverso():** Utiliza la instrucción yield return comenzando en la Cola para iterar eficientemente hacia atrás sin consumir memoria adicional.
- **GetEnumerator():** Implementa la interfaz IEnumerable<T> para permitir la búsqueda por jugador o por tipo mediante recorridos lineales.

4. Propiedades del Jugador: Lista Simplemente Enlazada (ListaSimple<T>)

- **Definición y Estructura:** Es una estructura lineal unidireccional formada por nodos simples (NodoSimple<T>). Cada nodo almacena la referencia al objeto del dato y un puntero único hacia el nodo Siguiente.
- **Nodo Asociado:** NodoSimple<Propiedad>.
- **Justificación de Uso en el Proyecto:** Cada objeto de tipo Jugador requiere mantener su propio inventario individual de inmuebles adquiridos:
 1. **Bajo Consumo de Memoria:** Al requerir únicamente un puntero unidireccional por nodo, es ideal para colecciones pequeñas y de tamaño dinámico.
 2. **Gestión de Modificaciones:** Permite iterar sobre el patrimonio del jugador para calcular su valor neto al final de la partida, así como eliminar o transferir propiedades de manera sencilla si un jugador cae en bancarrota.
- **Operaciones Principales e Implementación:**
 - Agregar(T dato): Conecta una nueva propiedad adquirida al final de la lista del jugador.
 - Eliminar(T dato): Recorre los nodos buscando la propiedad a retirar, reajustando el puntero Siguiente del nodo previo para desvincularla en caso de pérdida o bancarrota.

- **Manual de Ejecución del Sistema**

Este manual describe los pasos necesarios para compilar, configurar e iniciar el sistema completo de **Monopoly TEC**, incluyendo el servidor central, la aplicación cliente (GUI) y la integración del módulo de hardware Arduino (Lector RFID RDM6300, botones y displays de 7 segmentos).

1. Requisitos del Sistema Software Requerido:

- **.NET SDK 10.0** (o superior) instalado en el equipo.
- **Arduino IDE** (para compilar y cargar el firmware del hardware).
- Drivers de puerto serie USB (CH340 / FTDI o estándar según la placa Arduino utilizada).

Hardware Requerido (Opcional si se usa simulación):

- 1x Placa Arduino (Uno / Nano / Mega).
- 1x Lector RFID RDM6300 (conectado a los pines D2/D3 mediante SoftwareSerial).
- 2x Displays de 7 segmentos (conectados a los pines de salida digital/analógica).
- 1x Pulsador / Botón físico (conectado al pin D6 con resistencia INPUT_PULLUP).
- Tarjetas o llaveros RFID (125 kHz).

2. Configuración y Carga del Hardware (Arduino)

1. Conecte la placa Arduino a la computadora mediante el cable USB.
2. Abra el archivo **Hardware.ino** en el Arduino IDE.
3. Seleccione la placa y el puerto COM asignado (ej. COM3 o /dev/ttyUSB0).
4. Verifique que la velocidad de comunicación del monitor serie coincida con **9600 baudios**.
5. Compile y descargue el programa a la placa Arduino.

Nota: La placa enviará comandos por el puerto serial usando los formatos:

- DADOS:x,y al presionar el botón físico (ej. DADOS:4,2).
- RFID:xxxxxxxx al aproximar un tag/tarjeta (ej. RFID:0001823940).

3. Compilación y Ejecución del Servidor Central

El servidor coordina la partida, maneja los turnos, valida la lógica de negocio y mantiene el registro de transacciones.

1. Abra una terminal o consola de comandos en la carpeta raíz de la solución.
2. Diríjase al directorio del servidor: **Bash cd Server**
3. Ejecute el proyecto indicando opcionalmente el límite máximo de turnos (por defecto es 40): **Bash dotnet run -- 40**
4. El servidor se iniciará escuchando en el **Puerto TCP 5000**.
5. **Controles de Consola del Servidor:**
 - **Presionar ENTER (fase de espera):** Inicia la partida manualmente (requiere un mínimo de 2 jugadores con tarjeta registrada).
 - **Partida de 4 jugadores:** Si se conectan 4 jugadores y todos registran su tarjeta RFID, la partida **inicia automáticamente**.
 - **Presionar ENTER (con la partida en curso):** Detiene el servidor y finaliza la sesión exportando las transacciones.

4. Compilación y Ejecución del Cliente (GUI)

Cada jugador ejecuta una instancia de la aplicación cliente para interactuar con el tablero de juego.

1. Abra una nueva ventana de consola por cada jugador.
2. Navegue al directorio de la interfaz gráfica del cliente: **Bash cd ClientGUI**
3. Inicie el cliente: **Bash dotnet run**
4. **Pantalla de Conexión:**
 - **Nombre de Jugador:** Ingrese su alias/identificador.
 - **IP Servidor:** Ingrese 127.0.0.1 (si el servidor corre localmente) o la IP de la red local donde se ejecuta el servidor.
 - **Puerto:** 5000.

- **Puerto Serial COM:** Seleccione el puerto asignado al Arduino (ej. COM3). Si no cuenta con el hardware conectado, marque la opción **Modo Simulación**.

5. Presione **Conectar**.

5. Flujo Completo de la Partida (Paso a Paso)

1. Fase de Conexión:

- Inicie el Servidor central. o Ejecute entre 2 y 4 instancias del Cliente GUI e ingrese los nombres de los jugadores.

2. Fase de Registro de Tarjetas RFID:

- Antes de iniciar la partida, cada jugador debe vincular su tarjeta RFID. o Acerque su tarjeta física al lector RFID de la placa Arduino o bien utilice la función de simulación en pantalla digitando/generando el ID de la tarjeta.

3. Inicio de Partida:

- Con 4 jugadores listos y registrados, el servidor arranca la partida automáticamente. Si hay entre 2 y 3 jugadores, el organizador presiona ENTER en la consola del Servidor para iniciar.

4. Desarrollo del Turno:

- Cuando la interfaz indique ES TU TURNO:
 1. **Lanzar Dados:** Presione el botón físico del Arduino o el botón visual *Tirar Dados*.
 2. **Acción/Compra:** Si cae en una propiedad disponible, el sistema le ofrecerá comprarla.
 3. **Confirmación con Tarjeta:** Para confirmar la acción de compra, rechazo, pago o finalización de turno, acerque su tarjeta RFID al lector cuando la consola/GUI se lo solicite.

5. **Fin de Partida y Exportación:**

- Al finalizar la partida (por bancarrota de los demás jugadores o al alcanzar el límite máximo de turnos), el servidor generará automáticamente el archivo de log:
- **transacciones_partida.txt** (ubicado en el directorio de ejecución del Servidor).

- **Protocolo de Comunicación TCP**

1. INTRODUCCIÓN

Este documento describe el protocolo de comunicación entre el **servidor** (el banco, que resguarda el estado oficial de la partida) y los **clientes** (uno por jugador). Los mensajes viajan a través del protocolo **TCP**, encapsulados en **JSON**, a razón de un mensaje por línea.

- **Flujo del Cliente al Servidor:** Cliente — Mensaje JSON + "\n" —► Servidor
- **Flujo del Servidor al Cliente:** Cliente ◀— Mensaje JSON + "\n" — Servidor

El cliente **nunca** modifica saldos, posiciones, propiedades, turnos, dados ni transacciones de manera autónoma. Su función es solicitar acciones; el servidor se encarga de validar y responder.

Las definiciones del protocolo están centralizadas en Network/Protocolo.cs, compartido por los proyectos Client, ClientGUI y Server.

2. TECNOLOGÍAS UTILIZADAS

- **TCP:** Uso de TcpClient y TcpListener en el puerto **5000**.
- **NetworkStream:** Canal para el envío y recepción continua de bytes.
- **UTF-8:** Codificación estándar para la conversión de texto a bytes.
- **JSON (System.Text.Json):** Formato de intercambio de datos.
- **Threading:** Escucha continua en segundo plano (un hilo por cliente en el servidor y un hilo dedicado en ClienteTcp).

3. FORMATO DE UN MENSAJE

Todo mensaje que circula por la red utiliza el envoltorio Mensaje:

C#

```
public class Mensaje
{
    public TipoMensaje Tipo { get; set; }
    public object? Datos { get; set; }
}
```

- **Tipo:** Especifica la intención del mensaje. En el JSON se serializa **como texto** (ejemplo: "CONECTAR"), gracias al atributo `[JsonConverter(typeof(JsonStringEnumConverter))]`.
- **Datos:** Carga útil del mensaje. Toma el valor null si el tipo de mensaje no requiere información adicional.

Los nombres de las propiedades en JSON coinciden exactamente con los definidos en C# (Tipo, Datos, IdJugador, etc.).

Ejemplo en tránsito por la red:

```
{"Tipo":"TIRAR_DADO","Datos":{"Dado1":3,"Dado2":5}}\n
```

Delimitador de trama

Debido a que TCP opera como un flujo continuo de bytes sin límites explícitos de mensaje, **cada mensaje finaliza estrictamente en un salto de línea (\n)**. Dado que el JSON generado no incluye saltos de línea internos, la regla es: *una línea equivale a un mensaje*.

4. TIPOS DE MENSAJES

4.1 Mensajes de Cliente a Servidor

Tipo	Datos (Payload)	Requiere Tarjeta	Descripción
CONECTAR	DatosConectar	No	Solicita unirse a la partida actual.
REGISTRAR_TARJETA	DatosTarjeta (IdTarjeta, NombreJugador)	No	Vincula una tarjeta RFID a un jugador (exclusivo sala de espera).
TIRAR_DADO	DatosTirarDado	No	Envía los valores obtenidos por el módulo físico de dados.
COMPRAR_PROPIEDAD	null	Sí	Manifiesta el deseo de comprar la propiedad ofrecida.
NO_COMPRAR	null	Sí	Rechaza la oferta de compra de la propiedad.
PAGAR_DEUDA	null	Sí	Efectúa el pago de una deuda pendiente (alquiler, impuesto o carta).
TERMINAR_TURNOS	null	Sí	Finaliza el turno del jugador actual.

Tipo	Datos (Payload)	Requiere Tarjeta	Descripción
CONSULTAR_TRANSACCIONES	null	Sí	Solicita la bitácora o historial de transacciones.
TARJETA_RFID	DatosTarjeta (solo IdTarjeta)	—	Notifica la lectura de una tarjeta para confirmar una acción pendiente.
DESCONECTAR	null	No	Abandona la sala de espera o la partida en curso.

4.2 Mensajes de Servidor a Cliente

Tipo	Datos (Payload)	Destinatarios	Descripción
CONEXION_ACEPTADA	DatosConexionAceptada	Emisor	Confirma el ingreso y asigna el ID correspondiente.
CONEXION_RECHAZADA	DatosConexionRechazada	Emisor	Detalla el motivo del rechazo y procede a cerrar el socket.
ACTUALIZAR_ESTADO	DatosActualizarEstado	Todos	Difunde el estado

Tipo	Datos (Payload)	Destinatarios	Descripción
			completo de todos los jugadores.
TU_TURN0	DatosTuTurno	Todos	Notifica el cambio de turno al jugador correspondiente.
RESULTADO_DADO	DatosResultadoDado	Todos	Informa el valor de los dados lanzados.
OFERTA_COMPRA	DatosOfertaCompra	Afectado	Notifica que la casilla actual est disponible para compra.
HISTORIAL_TRANSACCIONES	DatosHistorialTransacciones	Solicitante	Entrega la lista histrica de transacciones.
EVENTO	DatosEvento	Todos / Uno	Transmite un mensaje informativo (con IdJugador = 0 es un evento global).
ERROR	DatosError	Afectado	Notifica un fallo de

Tipo	Datos (Payload)	Destinatarios	Descripción
			validación o infracción de regla.
			Declara la conclusión de la partida e informa el ganador.
FIN_JUEGO	DatosFinJuego	Todos	

Nota: No existe el comando CONSULTAR_ESTADO. El servidor transmite ACTUALIZAR_ESTADO de forma automática ante cada modificación en el juego.

5. ESTRUCTURA DE LOS DATOS (PAYLOADS)

- **DatosConectar:** { "Tipo": "CONECTAR", "Datos": { "Nombre": "Cristopher" } }(Si el nombre está vacío, el servidor asigna "JugadorN". Los nombres no distinguen mayúsculas).
- **DatosConexionAceptada:** { "Tipo": "CONEXION_ACEPTADA", "Datos": { "IdJugador": 1, "NumeroCasillas": 40, "Jugadores": [] } }
- **DatosConexionRechazada:** { "Tipo": "CONEXION_RECHAZADA", "Datos": { "Motivo": "La partida esta llena" } }
- **JugadorEstado:** { "Id": 1, "Nombre": "Ana", "Posicion": 12, "Saldo": 1340, "Propiedades": [1, 3], "EnBancarrota": false, "TieneTarjeta": true }
- **DatosTuTurno:** { "Tipo": "TU_TURNO", "Datos": { "IdJugador": 2 } }
- **DatosTirarDado:** { "Tipo": "TIRAR_DADO", "Datos": { "Dado1": 3, "Dado2": 4 } }
- **DatosResultadoDado:** { "Tipo": "RESULTADO_DADO", "Datos": { "IdJugador": 1, "Dado1": 4, "Dado2": 6 } }
- **DatosOfertaCompra:** { "Tipo": "OFERTA_COMPRA", "Datos": { "IdCasilla": 3, "NombrePropiedad": "Sapriisa", "Precio": 60 } }

- **DatosHistorialTransacciones:** Transporta una lista de objetos TransaccionInfo (ID, FechaHora, Turno, Tipo, JugadorOrigen, JugadorDestino, Monto, Descripcion). Tipos válidos: COMPRA_PROPIEDAD, PAGO_ALQUILER, PAGO_BANCO, PAGO_ENTRE_JUGADORES, GANANCIA_EVENTO, PERDIDA_EVENTO, PREMIO_INICIO.
- **DatosEvento:** { "Tipo": "EVENTO", "Datos": { "IdJugador": 0, "Descripcion": "Ana avanza 7 casillas" } }
- **DatosError:** { "Tipo": "ERROR", "Datos": { "Mensaje": "No es tu turno" } }
- **DatosFinJuego:** { "Tipo": "FIN_JUEGO", "Datos": { "IdGanador": 2 } }

6. CICLO DE VIDA DE UNA PARTIDA

6.1 Sala de Espera

1. El cliente transmite el mensaje CONECTAR.
2. El servidor evalúa y responde con CONEXION_ACEPTADA o CONEXION_RECHAZADA. En caso de aceptación, transmite ACTUALIZAR_ESTADO y EVENTO a los clientes conectados.
3. Cada participante asocia su tarjeta RFID ejecutando REGISTRAR_TARJETA.
4. La partida inicia bajo dos condiciones:
 - **Manual:** El administrador presiona **ENTER** en la consola del servidor (mínimo 2 jugadores con tarjeta registrada).
 - **Automática:** Se completan 4 jugadores y todos han registrado su tarjeta RFID.

6.2 Partida en Curso

Únicamente el jugador poseedor del turno activo puede realizar acciones. Secuencia del turno:

1. TIRAR_DADO —► Avance por el tablero —► Evaluación de casilla.
2. Interacción según casilla (OFERTA_COMPRA, pago de deuda, evento o casilla neutra).
3. Confirmación de acción vía tarjeta RFID (COMPRAR_PROPIEDAD, NO_COMPRAR, PAGAR_DEUDA).

4. Cierre de turno mediante TERMINAR_TURNO + tarjeta RFID —► Emisión de TU_TURNO al siguiente jugador.

6.3 Fin de la Partida

La partida concluye cuando queda un solo jugador activo (por quiebra de los demás) o al alcanzar el límite de turnos preestablecido. Al finalizar, el servidor emite FIN_JUEGO y exporta la bitácora al archivo transacciones_partida.txt.

7. MANEJO DE DADOS FÍSICOS

El módulo físico de Arduino gestiona la lectura de los dados. **El servidor no genera valores aleatorios**, únicamente valida que los dados recibidos estén en el rango de 1 a 6 y aplicables al jugador que posee el turno.

8. ACCIONES CON TARJETA (MECANISMO DE DOS PASOS)

Para garantizar la seguridad en dispositivos compartidos, las acciones financieras y de turno requieren confirmación RFID en dos etapas:

[Cliente] — 1. Elige Acción (ej. COMPRAR_PROPIEDAD) —► [Servidor]

[Cliente] ◀— 2. Solicita RFID ("Acerca tu tarjeta...") — [Servidor]

[Hardware] — 3. Lectura RFID (TARJETA_RFID) —————► [Servidor]

[Cliente] ◀— 4. Ejecuta Acción y Actualiza Estado ——— [Servidor]

9. ORDEN DE PUBLICACIÓN DE MENSAJES

Con el fin de evitar parpadeos o desincronización en la interfaz gráfica (la cual limpia o redibuja elementos al recibir actualizaciones de estado), el servidor transmite las respuestas respetando estrictamente la siguiente jerarquía:

1. ACTUALIZAR_ESTADO (Difusión general).
2. TU_TURNO (En caso de cambio de turno).
3. RESULTADO_DADO (Si aplica a la acción).
4. EVENTO (Mensajes de texto acumulados).
5. FIN_JUEGO (Si se cumplió la condición de victoria).

10. MANEJO DE ERRORES Y VALIDACIONES

Causa del Error	Mensaje Retornado por el Servidor
Petición sin conexión previa	Primero debes conectarte
Formato de mensaje desconocido	Mensaje no reconocido
CONECTAR en cliente ya activo	Ya estas conectado
Petición con campos requeridos nulos	Faltan los valores de los dados / Faltan los datos de la tarjeta
Acción fuera de partida	La partida no ha iniciado / La partida ya termino
Jugador en bancarrota	Estas eliminado
Intento de acción fuera de turno	No es tu turno
Intento de doble tirada	Ya lanzaste los dados en este turno
Intento de compra sin casilla	No hay ninguna propiedad disponible para comprar
Saldo insuficiente	No tienes saldo suficiente para comprar <propiedad>
Intento de finalizar turno inválido	Debes lanzar los dados antes de terminar el turno
Intento de finalizar con deudas	Tienes una deuda pendiente, debes pagarla antes de terminar el turno
Lectura de tarjeta no registrada	Tarjeta no registrada

11. SERIALIZACIÓN Y LECTURA DE DATOS

Proceso de Envío:

Objeto Mensaje —► Serializar() —► JSON —► + "\n" —► Bytes UTF-8 —►
NetworkStream.Write()

Proceso de Recepción:

NetworkStream.Read() —► Acumulador (StringBuilder) —► Corte por "\n" —►
Deserializar() —► Instancia de Mensaje

12. INTEGRACIÓN CON HARDWARE ARDUINO

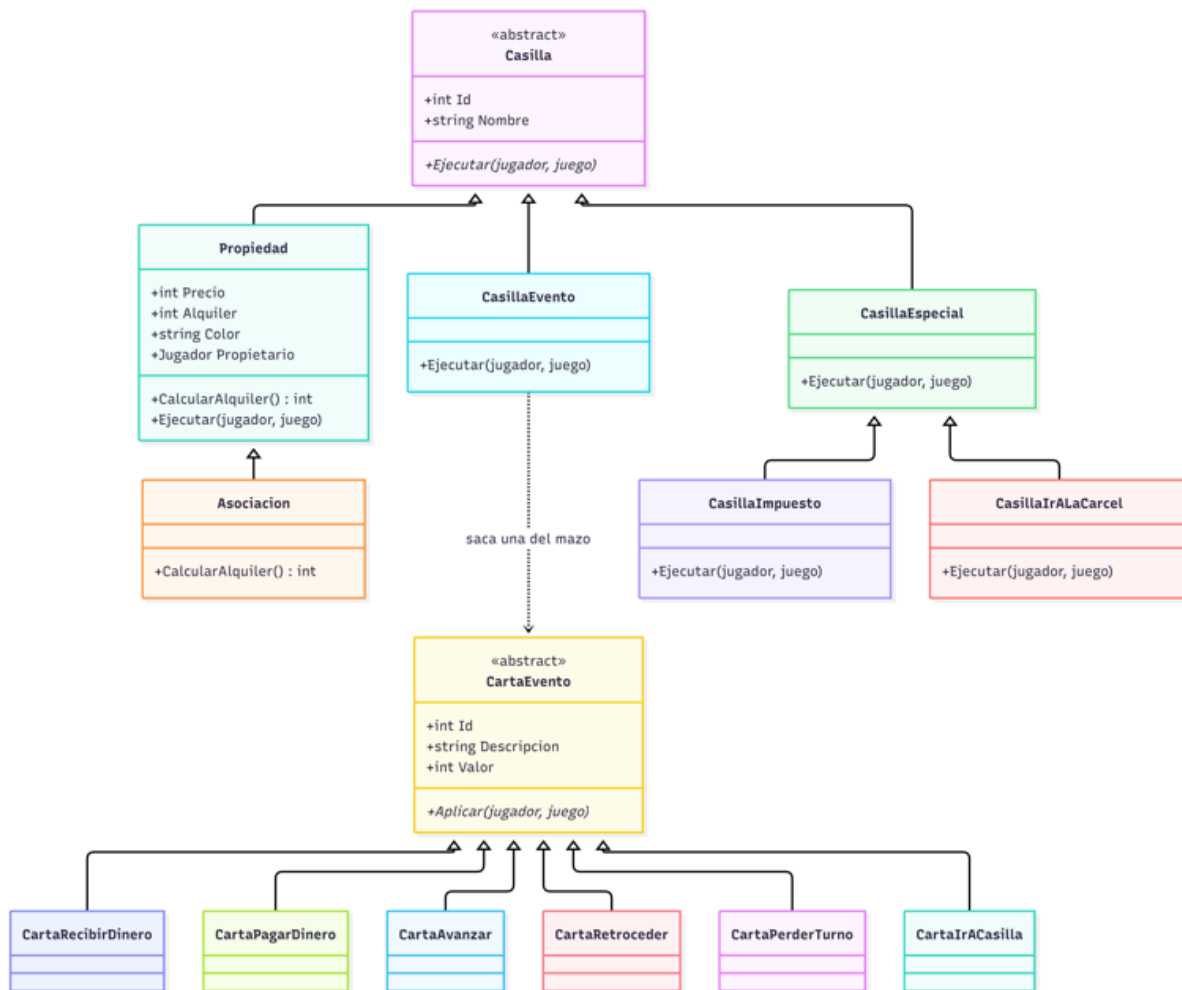
La clase DispositivoHardware lee el puerto serial (9600 baudios) y mapea las líneas recibidas hacia el protocolo del sistema:

- **Trama Serial:** DADOS:3,5 —► **Acción C#:** Emite TIRAR_DADO (Dado1 = 3, Dado2 = 5).
- **Trama Serial:** RFID:0A1B2C3D4E —► **Acción C#:** Emite REGISTRAR_TARJETA (si está en la sala de espera) o TARJETA_RFID (si está en partida).

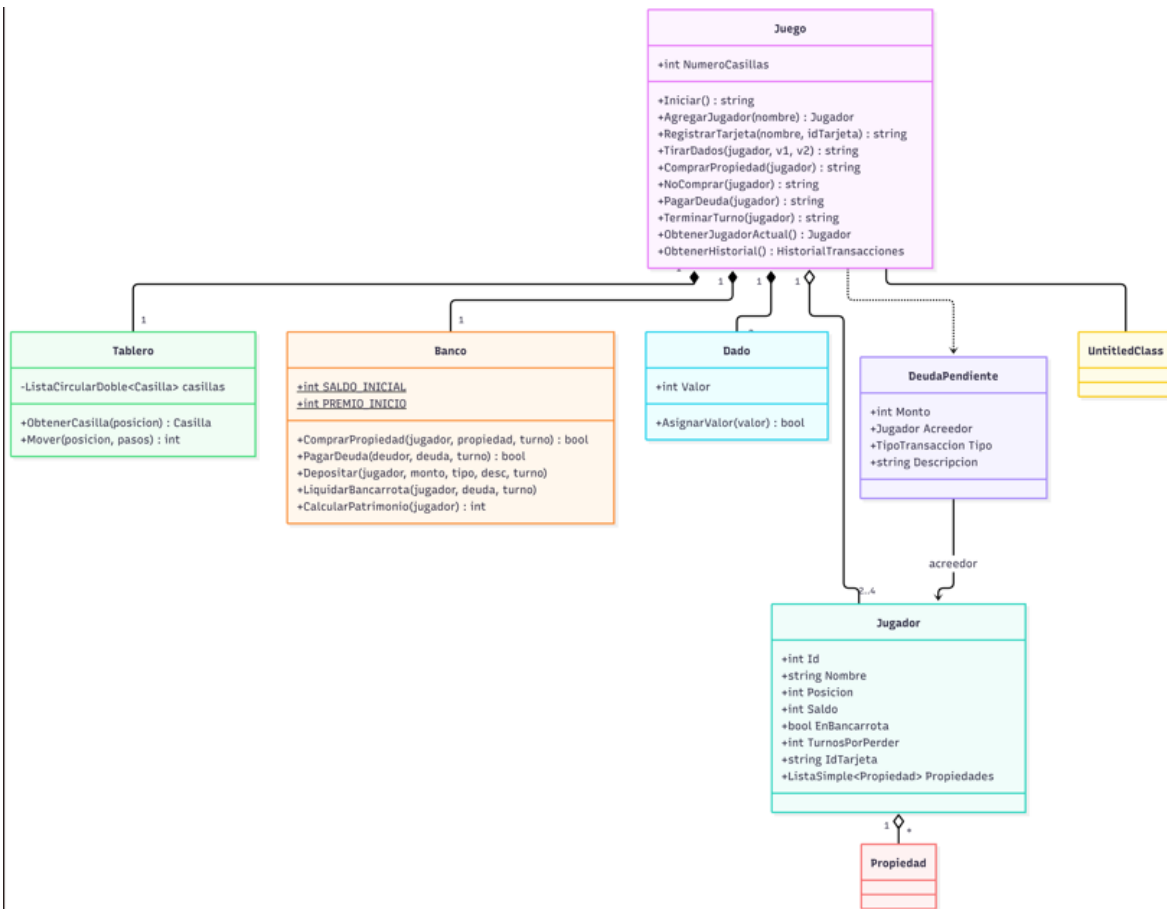
13. REGLAS FUNDAMENTALES DEL PROTOCOLO

1. Todo paquete transmitido por la red debe instanciar la clase Mensaje.
2. La propiedad Tipo debe serializarse como el nombre del enumerado en formato texto.
3. Todo mensaje debe finalizar obligatoriamente con el carácter de salto de línea (\n) y codificarse en **UTF-8**.
4. El servidor se establece como la **única fuente de verdad** del estado del juego; el cliente actúa como un terminal de presentación y comandos.
5. El cliente y el servidor deben compilarse utilizando exactamente el mismo archivo fuente Network/Protocolo.cs.

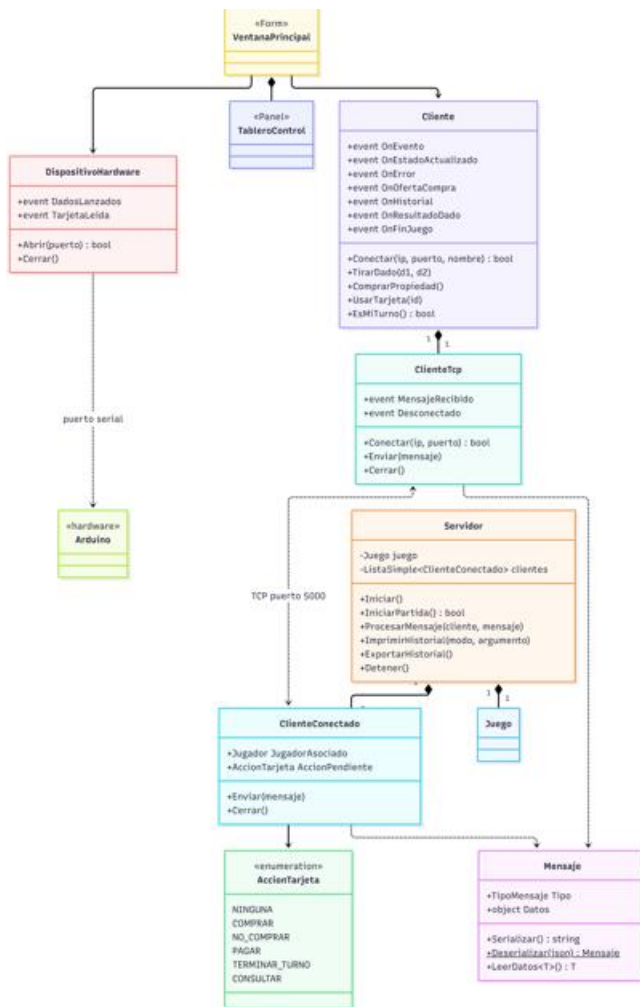
- UML casillas y cartas



- UML núcleo del juego



- UML red servidor y clientes



- UML transacciones y estructuras de datos

